



Spendly App Knowledge Base

SmartExpense AI — Technical Architecture, Ingestion Pipelines & System Constraints

1. Platform Overview

Spendly (also known as **SmartExpense AI**) is a premium, intelligent, offline-first personal finance tracker designed to eliminate the friction of managing budgets and logging transactions. It integrates local-first SQLite performance with real-time Supabase cloud sync and Google Gemini AI services.

- Frontend: React Native
- Expo & Expo Router
- TypeScript
- Zustand State Engine
- SQLite Local Cache
- Supabase Backend
- Google Gemini AI

Frontend Architecture

React Native, Expo, TypeScript, Expo Router (file-based routing), and Zustand global state stores optimized for rapid rendering and offline resilience.

Backend & AI Engine

Supabase Auth, PostgreSQL, Storage, Edge Functions. Google Gemini AI handles OCR, category mapping, vector embeddings, and RAG synthesis.

2. Core Features & Capabilities

Feature	Supported Input	Description & Processing Logic
Gemini OCR Scanner	Physical Receipts	Extracts merchant name, total value, tax, transaction date, payment method, and individual line items into structured JSON.
UPI Auto-Detection	GPay, PhonePe, Paytm	Parses screenshot confirmation images to instantly detect payment amounts, bank reference numbers, dates, and merchant names.
AI Category Classifier	Merchant Text String	Automatically maps merchant names to standard spending categories in milliseconds (e.g., <i>Starbucks</i> → Food).
Interactive Analytics	Ledger Database	Custom bar/pie charts mapping month-over-month spending trends with flexible category and date range filters.
Multi-Currency Converter	Foreign Currencies	Local caching of exchange rates allows entering foreign expenses with real-time conversion to default ledger currency.

3. Supported Spending Categories

Spendly maps all incoming expenses to one of the following 12 structured standard categories:

1. 🍔 **Food:** Restaurants, cafes, food deliveries (Starbucks, Swiggy, Zomato, McDonald's).

2. 🛒 **Grocery:** Supermarkets and wholesale outlets (Walmart, Target, Costco, local grocers).

3. ✈️ **Travel:** Flights, taxi rides, trains, hotel bookings, ride-hailing (Uber, Lyft, Airbnb).

7. 🗑️ **Bills:** Utilities, internet subscriptions, phone recharges, insurance (AT&T, electricity).

8. 🏥 **Health:** Medical checkups, pharmacies, gym memberships, clinical charges (CVS).

9. 🏠 **Rent:** Monthly housing leases, mortgages, and landlord payouts.

- 4. 🛢️ **Fuel:** Gas stations and charging stations (Shell, Chevron, ExxonMobil).
- 5. 🛍️ **Shopping:** Online retail, apparel, and gadgets (Amazon, Nike, Zara, Apple Store).
- 6. 🎬 **Entertainment:** Movies, streaming services, video games (Netflix, Spotify, Steam).

- 10. 🇪🇺 **EMI:** Loan installments, financing interest, and credit card payments.
- 11. 📖 **Education:** Courses, school/college fees, book purchases (Udemy, Coursera).
- 12. 📦 **Other:** General fallback for miscellaneous or uncategorized items.

4. Offline-First Architecture & Syncing

Local-First Cache Guarantee

All transactions, settings, and budgets are written directly to a highly optimized local SQLite database cache, enabling instant screen loads and uninterrupted transaction logging offline.

Cloud Synchronization & Network Monitoring:

- **Batch Online Sync:** When connectivity is re-established, offline changes are automatically batch-synchronized with the remote Supabase PostgreSQL database.
- **Graceful Interruptions:** Zustand state engine coupled with hardware network listeners monitors connection transitions. If connection drops during LLM streaming or cloud requests, the app gracefully pauses and alerts the user.

5. Spendly AI Hub & RAG Chatbot Architecture

The **Spendly AI Chat Assistant** is a Retrieval-Augmented Generation (RAG) agent that answers user queries using uploaded documents combined with LLM reasoning.

Supported File Formats & Parsers

- **PDFs (.pdf):** Parsed page-by-page using the edge-native `unpdf` library in the cloud.
- **Word Documents (.docx):** Parsed using the `mammoth` OpenXML parser library.
- **Spreadsheets (.xlsx , .csv):** Parsed and chunked by row using SheetJS (`xlsx`). Rows are formatted as standalone Q&A blocks to maintain ledger context.
- **Text & Markdown (.txt , .md):** Decoded and parsed natively.

End-to-End RAG Pipeline Flow

1. **Document Upload:** File selected in app → uploaded to private Supabase Storage bucket (`documents`).
2. **Metadata Registration:** File entry created in `public.documents` database table.
3. **Ingestion Trigger:** `ingest-document` Edge Function downloads file, parses text, splits into semantic chunks with a sliding window (preserving headings), generates 768-dimension vectors via Google `gemini-embedding-001`, and stores them in `document_chunks`.
4. **Query & Context Retrieval:** User sends prompt → `chat` Edge Function generates query vector → runs **Cosine Similarity & Keyword Hybrid Search** against vector index → fetches top 5 matching text chunks.
5. **LLM Synthesis & SSE Streaming:** Top 5 chunks appended to system prompt → dispatched to `gemini-2.5-flash` model → streamed back as Server-Sent Events (SSE) via a fragmented packet buffer parser.

Strict RAG Citation & Fallback Protocol

- In-line references (e.g. `[1]`, `[2]`) link directly to interactive modals with metadata (Document Title, Page, or Row Number).

- **Strict Fallback Rule:** If requested information is absent from uploaded documents, the assistant **MUST** respond with exactly:
"I'm sorry, I cannot find that information in your uploaded documents."

6. Rate Limits & Observability

Metric / Policy	Constraint / Mechanism	Purpose & Impact
Message Frequency	Max 10 messages / minute	Prevents abuse and API flooding.
Daily Token Allowance	50,000 tokens / day (prompts + completions)	Controls operational expenses per user.
File Upload Limit	Maximum 10 MB per file	Ensures efficient memory management during ingestion.
Feedback Logging	Thumbs Up / Down logged to <code>chat_feedback</code>	Provides quality assurance and context evaluation.
Auto-Summarization	Triggered after 10 messages in a thread	Gemini compresses historical dialogue into a summary to reduce token usage.

7. Database Schema & PostgreSQL Tables

The backend features five core PostgreSQL tables supporting the Knowledge Base and RAG Assistant:

Table Name	Key Fields	Description
<code>public.documents</code>	<code>id, title, filename, storage_path, uploaded_by, file_type, version</code>	Stores document metadata and storage references.
<code>public.document_chunks</code>	<code>id, document_id, chunk_text, embedding (vector(768)), page_number, section, metadata</code>	Holds semantic chunks and Google Gemini embeddings.
<code>public.chat_conversations</code>	<code>id, title, user_id, summary, created_at, updated_at</code>	Stores top-level conversation thread records and auto-summaries.
<code>public.chat_messages</code>	<code>id, conversation_id, role ('user' 'assistant' 'system'), content, citations, token_usage</code>	Stores individual conversation messages, tokens, and citations.
<code>public.chat_feedback</code>	<code>id, message_id, is_positive, comment, created_at</code>	Tracks user ratings (thumbs up/down) for response evaluation.

8. Frequently Asked Questions (FAQ)

Q: Why does the chat assistant say it cannot find information?

A: The chatbot operates under strict RAG constraints. If you have not uploaded files containing the answers, or if the answers are not found in the semantic context chunks, the chatbot replies with: "I'm sorry, I cannot find that information in your uploaded documents."

Q: Can I run Spendly completely offline?

A: Yes! You can log expenses, track budgets, and view analytics offline. Your changes sync automatically when reconnected. However, AI features (Receipt Scanning, UPI Parsing, and AI Chat Assistant) require internet connectivity.

Q: Is there a file size limit for uploads?

A: Yes, the maximum supported upload limit is **10MB** per file.

Q: How do I delete my uploaded documents?

A: In the **Knowledge Base** tab, click the trash can icon next to the document. This removes the metadata, vector chunks, and the underlying file from Supabase Storage.

Q: What is the rate limit for chat?

A: Usage is capped at **10 messages per minute** and **50,000 total tokens per day** per user.